

Sistem Multi-Agent pentru Compararea Algoritmilor de Machine Learning

Berciu Georgiana, Malan Andreea

Contents

1	Introducere	3
2	Dataset-ul utilizat	3
3	Arhitectura sistemului multi-agent	4
3.1	DataLoaderAgent	4
3.2	PreprocessingAgent	4
3.3	ModelTrainerAgent	5
3.4	VisualizationAgent	5
4	Preprocesarea datelor	5
5	Algoritmii utilizati	6
5.1	Logistic Regression	6
5.2	Decision Tree	6
5.3	K-Nearest Neighbors	6
5.4	Support Vector Machine	6
5.5	Random Forest	6
5.6	MLP Neural Network	6
6	Optimizarea hiperparametrilor cu GridSearchCV	7
7	Metrici de evaluare	8
7.1	Accuracy	8
7.2	Precision	8
7.3	Recall	8
7.4	F1 Score	8
7.5	Overfitting Difference	8

8	Rezultate experimentale	9
8.1	Tabelul complet cu rezultatele obtinute	9
8.2	Top 10 cele mai bune combinatii	10
9	Analiza graficelor generale	11
9.1	Compararea tuturor combinatiilor dupa Test Accuracy	11
9.2	Compararea tuturor combinatiilor dupa F1 Score	12
9.3	Heatmap pentru Test Accuracy	13
9.4	Heatmap pentru F1 Score	14
9.5	Comparatie intre Test Accuracy si F1 Score	14
9.6	Comparatie intre Train Accuracy si Test Accuracy	15
9.7	Diferenta dintre Train Accuracy si Test Accuracy	16
10	Analiza performantelor pe fiecare dataset	16
10.1	Compararea modelelor pe dataset-ul Original	17
10.2	Compararea modelelor pe dataset-ul Normalized	17
10.3	Compararea modelelor pe dataset-ul MinMax Scaled	18
10.4	Compararea modelelor pe dataset-ul Standardized	19
10.5	Compararea modelelor pe dataset-ul Robust Scaled	19
11	Analiza performantelor pentru fiecare model	20
11.1	Performanta modelului Decision Tree pe fiecare tip de dataset	20
11.2	Performanta modelului Random Forest pe fiecare tip de dataset	21
11.3	Performanta modelului MLP Neural Network pe fiecare tip de dataset	21
11.4	Performanta modelului KNN pe fiecare tip de dataset	22
11.5	Performanta modelului SVM pe fiecare tip de dataset	23
11.6	Performanta modelului Logistic Regression pe fiecare tip de dataset	23
12	Matricea de confuzie pentru cel mai bun model	24
13	Insight-uri automate generate de sistem	25
14	Cel mai bun model identificat	26
15	Concluzii	26

1 Introducere

Machine Learning reprezinta una dintre cele mai importante ramuri ale inteligentei artificiale, fiind utilizata pentru rezolvarea problemelor de clasificare, regresie si predictie. In cadrul acestui proiect a fost implementat un sistem multi-agent care automatizeaza fluxul de lucru specific unui proiect de Machine Learning.

Scopul proiectului este compararea mai multor algoritmi de clasificare pe dataset-ul Titanic si identificarea modelului cu cea mai buna performanta. Sistemul realizeaza incarcarea datelor, preprocesarea acestora, antrenarea modelelor, optimizarea hiperparametrilor si generarea rezultatelor sub forma de tabele si grafice.

Obiectivele proiectului sunt:

- incarcarea si analiza automata a datelor;
- tratarea valorilor lipsa;
- codificarea variabilelor categorice;
- aplicarea mai multor metode de scalare;
- antrenarea mai multor algoritmi de clasificare;
- optimizarea hiperparametrilor folosind GridSearchCV;
- compararea rezultatelor obtinute;
- analiza fenomenului de overfitting.

2 Dataset-ul utilizat

Pentru realizarea experimentelor a fost utilizat dataset-ul Titanic. Acesta contine informatii despre pasagerii aflatii la bordul vasului Titanic si despre supravietuirea acestora.

Variabila tinta este **Survived**, unde:

- 0 = pasagerul nu a supravietuit;
- 1 = pasagerul a supravietuit.

Principalele caracteristici utilizate sunt prezentate in Tabelul 1.

Table 1: Caracteristici principale ale dataset-ului Titanic

Atribut	Descriere
Pclass	Clasa pasagerului
Sex	Genul pasagerului
Age	Varsta pasagerului
SibSp	Numar rude aflate la bord
Parch	Numar parinti/copii aflatii la bord
Fare	Costul biletului
Embarked	Portul de imbarcare
Survived	Variabila tinta

Dataset-ul este potrivit pentru o problema de clasificare binara, deoarece obiectivul este prezicerea supravietuirii unui pasager.

3 Arhitectura sistemului multi-agent

Sistemul este organizat pe mai multi agenti, fiecare avand un rol clar in procesul de Machine Learning. Aceasta abordare ofera modularitate, flexibilitate si posibilitatea extinderii proiectului.

3.1 DataLoaderAgent

Agentul DataLoaderAgent este responsabil pentru incarcarea dataset-ului si analiza initiala a datelor.

Responsabilitati:

- citirea fisierului CSV;
- afisarea dimensiunii dataset-ului;
- identificarea valorilor lipsa;
- separarea caracteristicilor de variabila tinta.

3.2 PreprocessingAgent

Agentul PreprocessingAgent pregateste datele pentru antrenarea modelelor.

Responsabilitati:

- completarea valorilor lipsa;
- transformarea variabilelor categorice;
- impartirea datelor in train si test;

- generarea mai multor variante ale dataset-ului.

3.3 ModelTrainerAgent

Agentul ModelTrainerAgent antreneaza modelele si optimizeaza hiperparametrii.

Responsabilitati:

- antrenarea algoritmilor de clasificare;
- aplicarea GridSearchCV;
- calcularea metricilor de performanta;
- salvarea rezultatelor obtinute.

3.4 VisualizationAgent

Agentul VisualizationAgent are rolul de a genera tabele si grafice pentru interpretarea rezultatelor.

Responsabilitati:

- generarea tabelului complet de rezultate;
- afisarea celor mai bune combinatii;
- generarea graficelor comparative;
- afisarea matricei de confuzie pentru cel mai bun model.

4 Preprocesarea datelor

Preprocesarea este o etapa importanta deoarece calitatea datelor influenteaza direct performanta modelelor.

In proiect au fost realizate urmatoarele operatii:

- tratarea valorilor lipsa;
- codificarea variabilelor categorice;
- impartirea datelor in set de antrenare si set de testare;
- aplicarea mai multor metode de scalare.

Au fost generate urmatoarele variante ale dataset-ului:

- Original Dataset;

- Standardized Dataset;
- MinMax Dataset;
- Robust Dataset;
- Normalized Dataset.

Scopul acestor variante este analizarea impactului metodelor de preprocesare asupra performantelor algoritmilor.

5 Algoritmii utilizati

În proiect au fost comparați mai mulți algoritmi de clasificare.

5.1 Logistic Regression

Logistic Regression este un model liniar utilizat frecvent pentru clasificare binară. Acesta estimează probabilitatea ca o observație să aparțină unei anumite clase.

5.2 Decision Tree

Decision Tree construiește un arbore de decizie pe baza atributelor dataset-ului. Este ușor de interpretat, dar poate fi predispus la overfitting.

5.3 K-Nearest Neighbors

KNN clasifică o observație pe baza celor mai apropiați vecini din setul de antrenare. Performanța acestuia poate fi influențată puternic de scalarea datelor.

5.4 Support Vector Machine

SVM caută un hiperplan optim care separă clasele. Este eficient pentru probleme de clasificare și poate obține rezultate bune pe dataset-uri complexe.

5.5 Random Forest

Random Forest este un algoritm de tip ansamblu, format din mai mulți arbori de decizie. Acesta reduce riscul de overfitting și poate obține performanțe ridicate.

5.6 MLP Neural Network

MLP este o rețea neuronală artificială care poate învăța relații complexe între date. Performanța sa depinde de arhitectura și de parametrii folosiți.

6 Optimizarea hiperparametrilor cu GridSearchCV

Pentru optimizarea modelelor a fost utilizata metoda GridSearchCV. Aceasta testeaza automat mai multe combinatii de hiperparametri si selecteaza configuratia cu cele mai bune rezultate.

Formula generala pentru alegerea hiperparametrilor optimi este:

$$\theta^* = \arg \max_{\theta \in \Theta} Score(\theta)$$

unde:

- Θ reprezinta multimea combinatiilor de hiperparametri;
- $Score(\theta)$ reprezinta performanta obtinuta pentru o anumita combinatie;
- θ^* reprezinta combinatia optima.

```
grid_search = GridSearchCV(  
    estimator=model,  
    param_grid=self.param_grids[model_name],  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)  
  
grid_search.fit(X_train, y_train)  
  
best_model = grid_search.best_estimator_  
self.best_models[(dataset_name, model_name)] = best_model  
  
print("Cei mai buni parametri:")  
print(grid_search.best_params_)
```

Figure 1: Utilizarea GridSearchCV pentru optimizarea hiperparametrilor

In aceasta secventa de cod este utilizata clasa GridSearchCV din biblioteca Scikit-Learn pentru optimizarea automata a hiperparametrilor modelelor de clasificare. Pentru fiecare algoritm sunt testate mai multe combinatii de parametri definite anterior in param_grid. Validarea se realizeaza prin metoda Cross Validation cu 5 fold-uri, iar criteriul de selectie este acuratetea. Dupa finalizarea procesului, este selectat modelul care obtine cea mai buna performanta, iar parametrii optimi sunt afisati utilizand atributul best_params_.

7 Metrici de evaluare

Pentru evaluarea modelelor au fost utilizate mai multe metrice.

7.1 Accuracy

Accuracy reprezinta proportia predictiilor corecte.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

7.2 Precision

Precision masoara cate dintre predictiile pozitive sunt corecte.

$$Precision = \frac{TP}{TP + FP}$$

7.3 Recall

Recall masoara capacitatea modelului de a identifica exemplele pozitive.

$$Recall = \frac{TP}{TP + FN}$$

7.4 F1 Score

F1 Score combina Precision si Recall.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

7.5 Overfitting Difference

Pentru analiza overfitting-ului a fost utilizata diferenta dintre Train Accuracy si Test Accuracy.

$$OverfittingDifference = TrainAccuracy - TestAccuracy$$

O valoare mica indica o generalizare buna, iar o valoare mare indica faptul ca modelul a memorat datele de antrenare.

8 Rezultate experimentale

8.1 Tabelul complet cu rezultatele obtinute

	Dataset	Model	Accuracy	Precision	Recall	F1 Score	Train Accuracy	Test Accuracy	Overfitting Difference	Overfitting Status	Recommendation	Best Parameters
0	Normalized	Random Forest	0.826816	0.816667	0.710145	0.759690	0.856742	0.826816	0.029926	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
1	MinMax Scaled	MLP Neural Network	0.826816	0.865385	0.652174	0.743802	0.846910	0.826816	0.020094	Generalizare bună	Modelul generalizează bine pe datele de test.	{'activation': 'relu', 'hidden_layer_sizes': (...
2	Standardized	KNN	0.815642	0.790323	0.710145	0.748092	0.844101	0.815642	0.028459	Generalizare bună	Modelul generalizează bine pe datele de test.	{'n_neighbors': 5, 'weights': 'uniform'}
3	Standardized	SVM	0.815642	0.833333	0.652174	0.731707	0.841292	0.815642	0.025650	Generalizare bună	Modelul generalizează bine pe datele de test.	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
4	Standardized	Logistic Regression	0.815642	0.846154	0.637681	0.727273	0.810393	0.815642	-0.005249	Generalizare bună	Modelul generalizează bine pe datele de test.	{'C': 0.01, 'solver': 'lbfgs'}
5	MinMax Scaled	SVM	0.815642	0.875000	0.608696	0.717949	0.841292	0.815642	0.025650	Generalizare bună	Modelul generalizează bine pe datele de test.	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
6	MinMax Scaled	KNN	0.810056	0.786885	0.695652	0.738462	0.873596	0.810056	0.063540	Posibil overfitting	Modelul poate fi ușor supraantrenat. Se recoma...	{'n_neighbors': 3, 'weights': 'uniform'}
7	Robust Scaled	SVM	0.810056	0.818182	0.652174	0.725806	0.860955	0.810056	0.050899	Posibil overfitting	Modelul poate fi ușor supraantrenat. Se recoma...	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
8	Original	Random Forest	0.810056	0.872340	0.594203	0.706897	0.852528	0.810056	0.042472	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
9	Standardized	Random Forest	0.810056	0.872340	0.594203	0.706897	0.852528	0.810056	0.042472	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
10	MinMax Scaled	Random Forest	0.810056	0.872340	0.594203	0.706897	0.852528	0.810056	0.042472	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
11	Robust Scaled	Random Forest	0.810056	0.872340	0.594203	0.706897	0.852528	0.810056	0.042472	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
12	MinMax Scaled	Logistic Regression	0.804469	0.774194	0.695652	0.732824	0.800562	0.804469	-0.003907	Generalizare bună	Modelul generalizează bine pe datele de test.	{'C': 1, 'solver': 'liblinear'}
13	Normalized	Decision Tree	0.798883	0.761905	0.695652	0.727273	0.837079	0.798883	0.038196	Generalizare bună	Modelul generalizează bine pe datele de test.	{'criterion': 'gini', 'max_depth': 5}
14	Robust Scaled	Logistic Regression	0.798883	0.789474	0.652174	0.714286	0.807584	0.798883	0.008702	Generalizare bună	Modelul generalizează bine pe datele de test.	{'C': 0.1, 'solver': 'lbfgs'}

Figure 2: Rezultatele obtinute pentru combinatiile model-dataset

Tabelul prezinta rezultatele obtinute pentru mai multe combinatii intre algoritmi de clasificare si variantele de dataset generate in etapa de preprocesare. Sunt afisate metrice precum Accuracy, Precision, Recall, F1 Score, Train Accuracy, Test Accuracy si Overfitting Difference.

Se observa ca cele mai bune rezultate sunt obtinute de modelele Random Forest, MLP Neural Network, KNN, SVM si Logistic Regression, in functie de metoda de preprocesare aplicata. Coloana Overfitting Status ajuta la identificarea modelelor care generalizeaza bine pe datele de test.

8.2 Top 10 cele mai bune combinatii

	Dataset	Model	Test Accuracy	F1 Score	Precision	Recall	Overfitting Difference	Overfitting Status	Best Parameters
0	Normalized	Random Forest	0.826816	0.759690	0.816667	0.710145	0.029926	Generalizare bună	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
1	MinMax Scaled	MLP Neural Network	0.826816	0.743802	0.865385	0.652174	0.020094	Generalizare bună	{'activation': 'relu', 'hidden_layer_sizes': (...
2	Standardized	KNN	0.815642	0.748092	0.790323	0.710145	0.028459	Generalizare bună	{'n_neighbors': 5, 'weights': 'uniform'}
3	Standardized	SVM	0.815642	0.731707	0.833333	0.652174	0.025650	Generalizare bună	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
4	Standardized	Logistic Regression	0.815642	0.727273	0.846154	0.637681	-0.005249	Generalizare bună	{'C': 0.01, 'solver': 'lbfgs'}
5	MinMax Scaled	SVM	0.815642	0.717949	0.875000	0.608696	0.025650	Generalizare bună	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
8	Original	Random Forest	0.810056	0.706897	0.872340	0.594203	0.042472	Generalizare bună	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
9	Standardized	Random Forest	0.810056	0.706897	0.872340	0.594203	0.042472	Generalizare bună	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
10	MinMax Scaled	Random Forest	0.810056	0.706897	0.872340	0.594203	0.042472	Generalizare bună	{'criterion': 'entropy', 'max_depth': 5, 'n_es...
11	Robust Scaled	Random Forest	0.810056	0.706897	0.872340	0.594203	0.042472	Generalizare bună	{'criterion': 'entropy', 'max_depth': 5, 'n_es...

Figure 3: Top 10 cele mai performante combinatii model-dataset

Acest tabel evidentiaza cele mai bune combinatii obtinute in urma experimentelor. Cea mai buna performanta este obtinuta de modelul Random Forest pe dataset-ul Normalized, cu o valoare Test Accuracy de aproximativ 0.8268 si F1 Score de aproximativ 0.7597.

De asemenea, modelul MLP Neural Network pe dataset-ul MinMax Scaled obtine aceeasi valoare pentru Test Accuracy, dar un F1 Score putin mai mic. Acest lucru arata ca alegerea modelului final nu trebuie facuta doar dupa Accuracy, ci si dupa F1 Score si diferenta de overfitting.

9 Analiza graficelor generale

9.1 Compararea tuturor combinatiilor dupa Test Accuracy

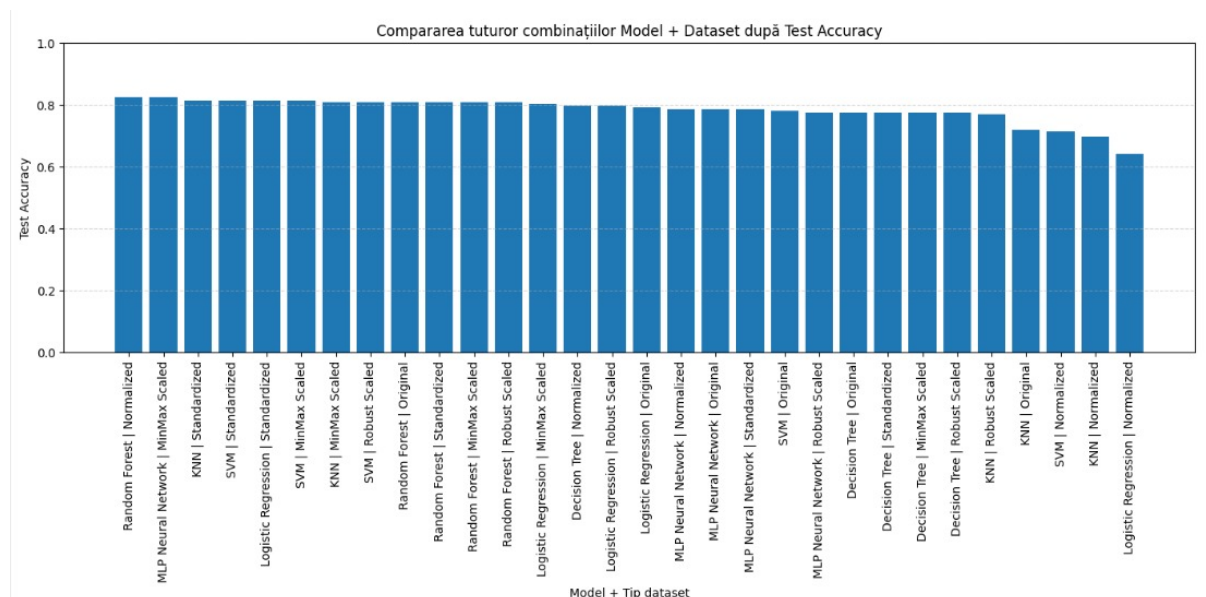


Figure 4: Compararea tuturor combinatiilor dupa Test Accuracy

Graficul prezinta toate combinatiile model-dataset ordonate dupa valoarea Test Accuracy. Se observa ca cele mai performante combinatii depasesc valoarea de 0.80, ceea ce indica o capacitate buna de clasificare.

Cele mai bune rezultate sunt obtinute de Random Forest pe dataset-ul Normalized si de MLP Neural Network pe dataset-ul MinMax Scaled. La finalul graficului se observa combinatiile cu performante mai slabe, unde valorile Accuracy scad sub 0.70.

9.2 Compararea tuturor combinatiilor dupa F1 Score

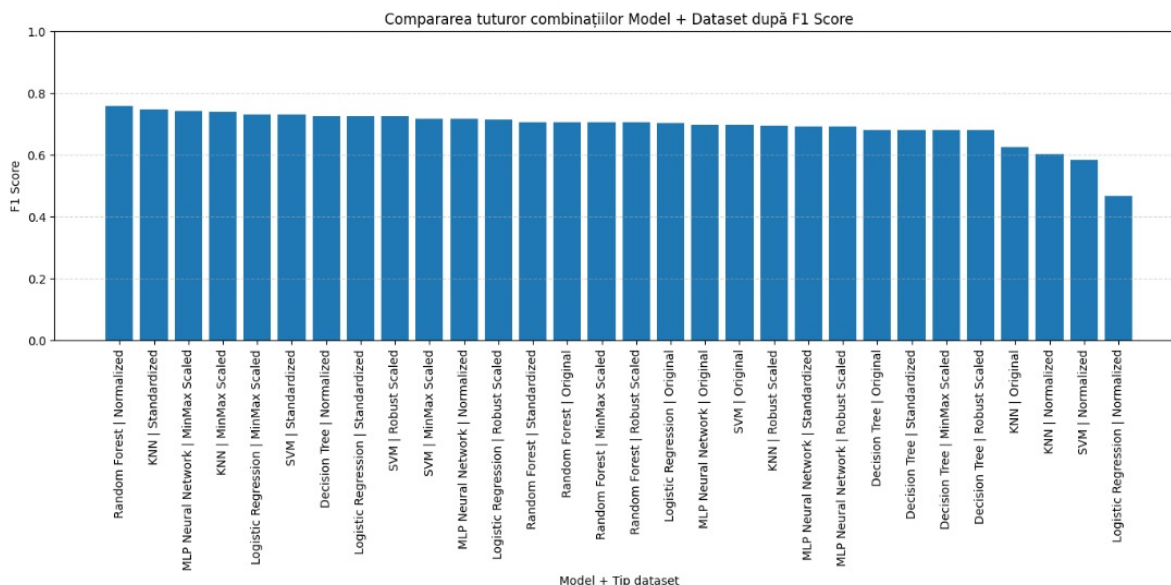


Figure 5: Compararea tuturor combinatiilor dupa F1 Score

Graficul compara modelele in functie de F1 Score. Aceasta metrica este importanta deoarece combina Precision si Recall, oferind o imagine mai echilibrata asupra performantei modelului.

Se observa ca Random Forest pe dataset-ul Normalized are cel mai mare F1 Score, ceea ce confirma faptul ca acest model nu doar clasifica bine in general, ci mentine si un echilibru bun intre predictiile corecte pozitive si identificarea exemplor pozitive.

9.3 Heatmap pentru Test Accuracy

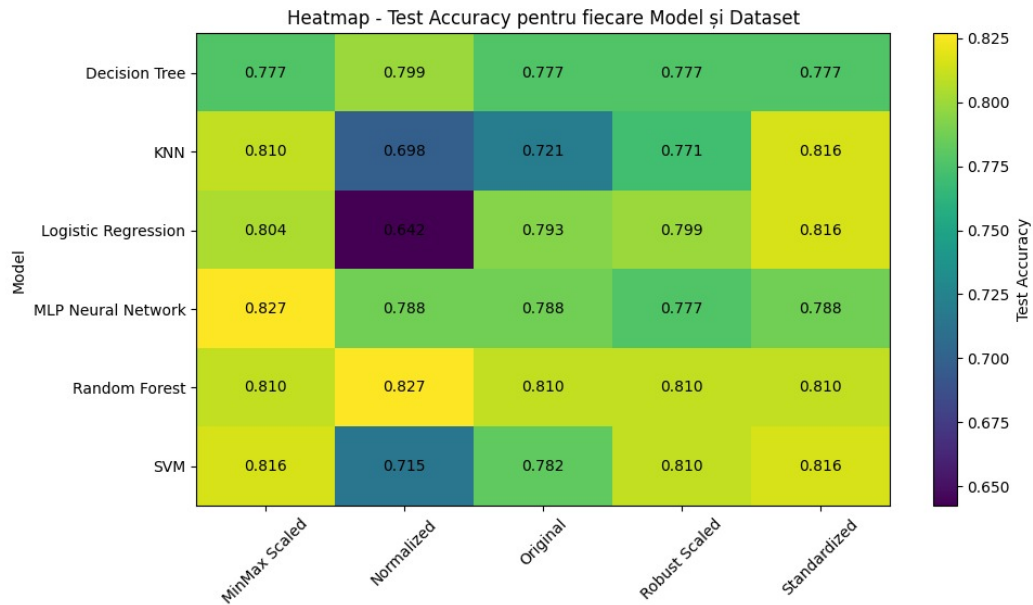


Figure 6: Heatmap pentru Test Accuracy

Heatmap-ul prezinta valorile Test Accuracy pentru fiecare combinatie dintre model si tipul de dataset. Culorile mai deschise indica performante mai bune.

Se observa ca Random Forest pe dataset-ul Normalized atinge una dintre cele mai mari valori, aproximativ 0.827. De asemenea, MLP Neural Network pe MinMax Scaled obtine o valoare similara. In schimb, Logistic Regression pe dataset-ul Normalized obtine o performanta mai slaba, ceea ce indica faptul ca aceasta metoda de preprocesare nu este potrivita pentru acest model.

9.4 Heatmap pentru F1 Score

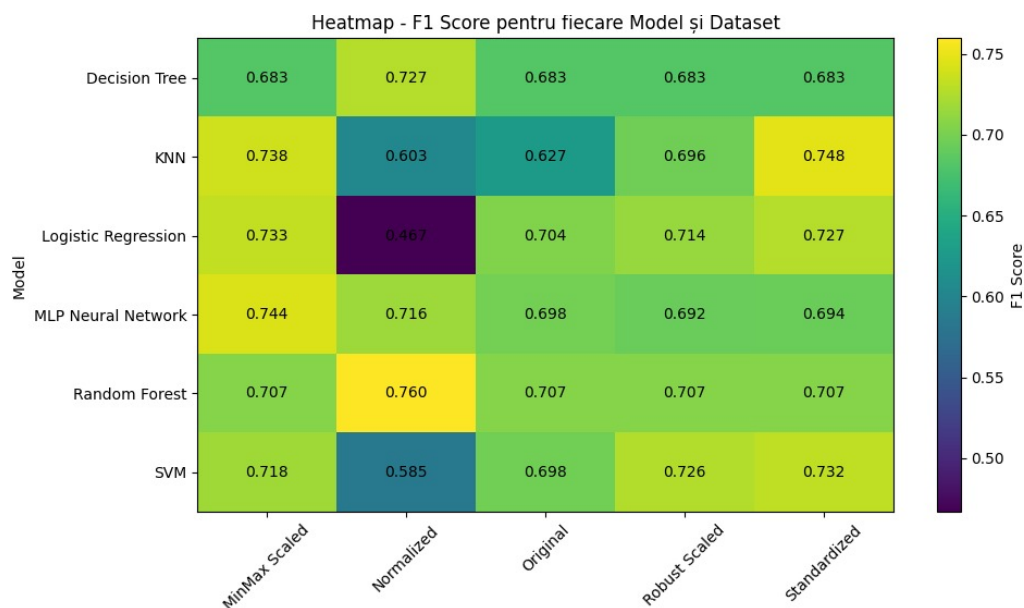


Figure 7: Heatmap pentru F1 Score

Acest heatmap evidentiaza valorile F1 Score pentru fiecare model si fiecare varianta de dataset. Random Forest pe dataset-ul Normalized obtine cea mai buna valoare, aproximativ 0.760.

Comparativ cu heatmap-ul pentru Accuracy, acest grafic arata mai clar diferentele dintre modele atunci cand se ia in considerare echilibrul dintre Precision si Recall. Logistic Regression pe Normalized are cea mai slaba valoare F1 Score, aproximativ 0.467.

9.5 Comparatie intre Test Accuracy si F1 Score

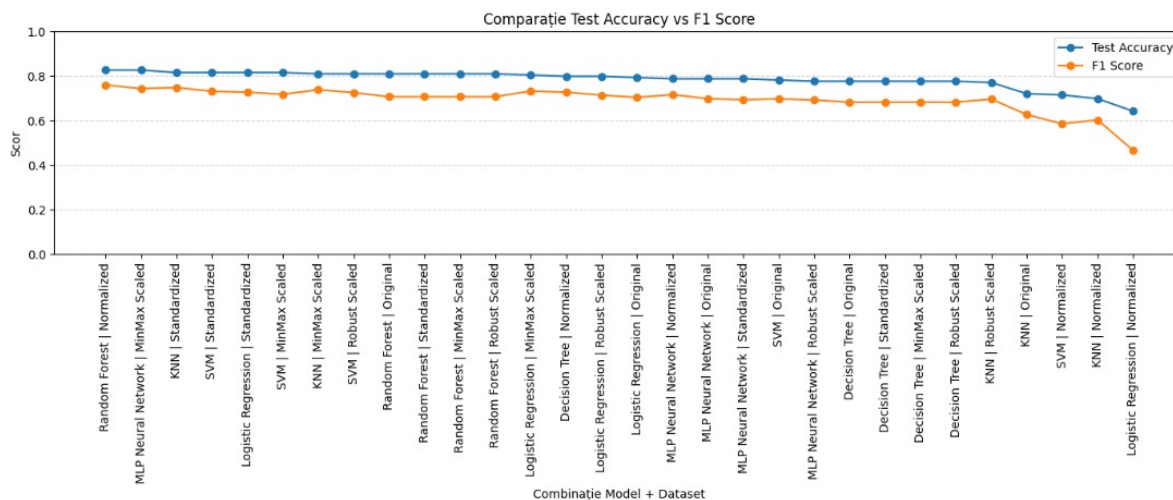


Figure 8: Comparatie intre Test Accuracy si F1 Score

Graficul compara simultan Test Accuracy si F1 Score pentru toate combinatiile testate. In general, Test Accuracy este mai mare decat F1 Score, ceea ce este normal deoarece F1 Score este o metrica mai stricta si tine cont de dezechilibrul dintre clase.

Diferentele mari dintre cele doua curbe pot indica situatii in care un model are acuratete buna, dar nu identifica la fel de bine clasa pozitiva. Din acest motiv, analiza trebuie facuta folosind mai multe metrice, nu doar Accuracy.

9.6 Comparatie intre Train Accuracy si Test Accuracy

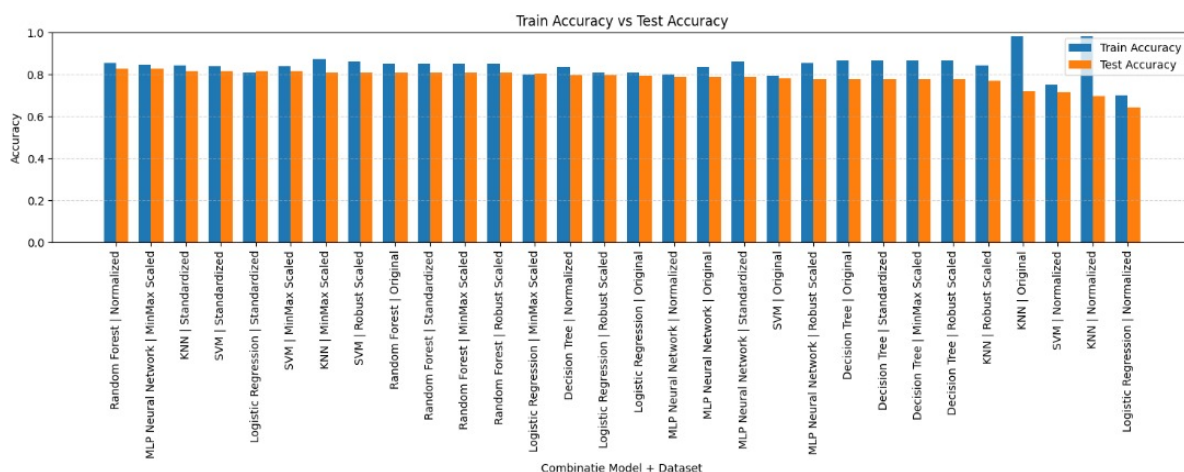


Figure 9: Comparatie intre Train Accuracy si Test Accuracy

Acest grafic este folosit pentru analiza fenomenului de overfitting. Daca Train Accuracy este mult mai mare decat Test Accuracy, inseamna ca modelul a invatat prea bine datele de antrenare si nu generalizeaza suficient de bine pe date noi.

Se observa ca majoritatea modelelor au valori apropiate intre Train Accuracy si Test Accuracy, ceea ce indica o generalizare buna. Totusi, pentru anumite combinatii, cum ar fi KNN pe dataset-ul Original, diferenta este mai mare, indicand un posibil overfitting.

9.7 Diferenta dintre Train Accuracy si Test Accuracy

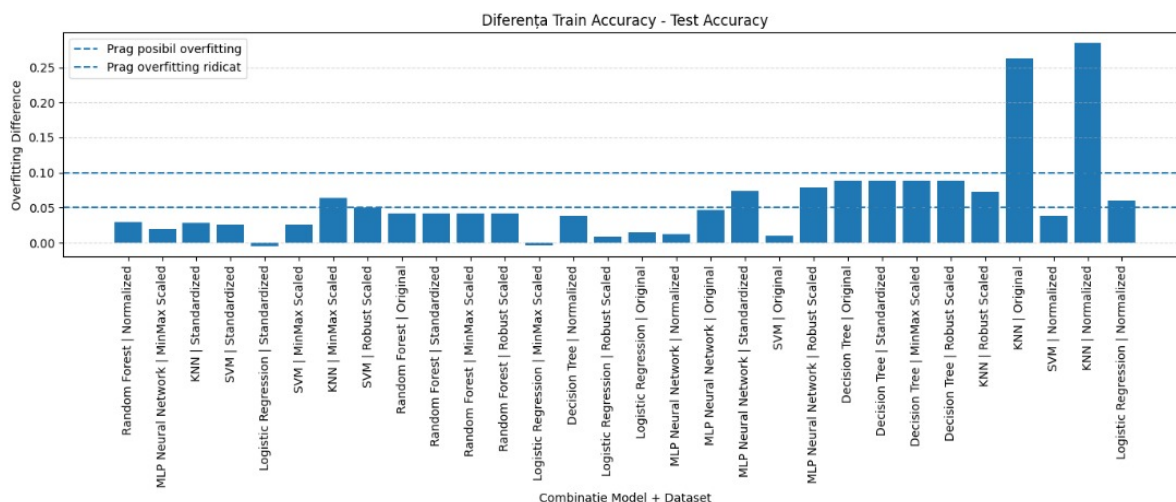


Figure 10: Diferenta Train Accuracy - Test Accuracy

Graficul prezinta diferenta dintre Train Accuracy si Test Accuracy pentru fiecare combinatie testata. Linia punctata de la 0.05 indica pragul pentru posibil overfitting, iar linia de la 0.10 indica un prag de overfitting ridicat.

Majoritatea combinatiilor se afla sub pragul de 0.05, ceea ce indica o generalizare buna. Exista insa cateva combinatii care depasesc pragurile, in special KNN pe dataset-ul Original si KNN pe dataset-ul Normalized, ceea ce arata ca aceste modele pot fi supraantrenate.

10 Analiza performantelor pe fiecare dataset

In aceasta sectiune sunt analizate performantele modelelor pe fiecare tip de dataset. Dataset-ul Original este prezentat primul pentru a evidentia performanta initiala, fara aplicarea metodelor de scalare. Ulterior sunt analizate variantele Normalized, MinMax Scaled, Standardized si Robust Scaled.

10.1 Compararea modelelor pe dataset-ul Original

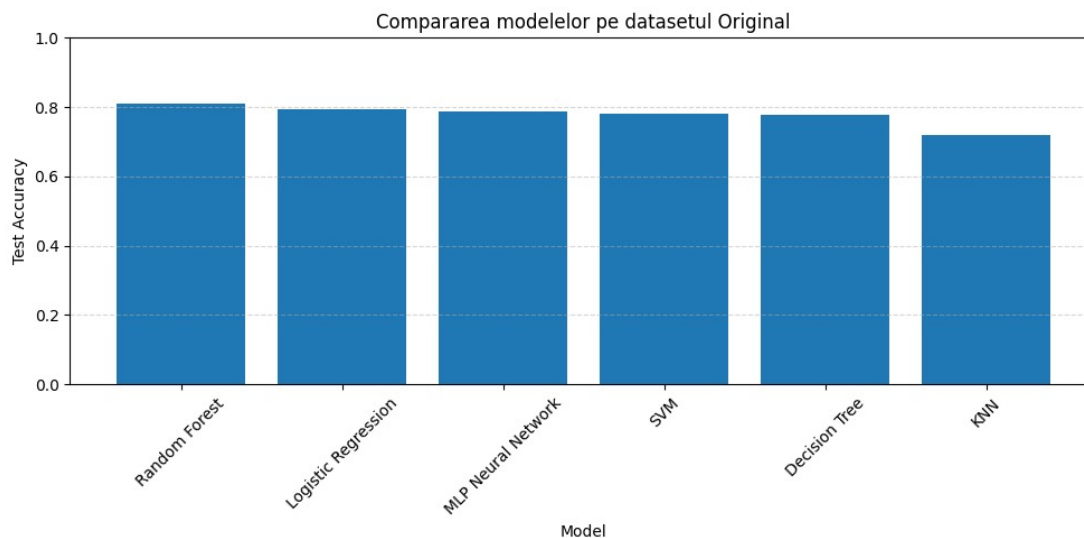


Figure 11: Compararea modelelor pe dataset-ul Original

Pe dataset-ul Original, Random Forest obtine cea mai buna performanta, cu o valoare Test Accuracy de aproximativ 0.81. Logistic Regression si MLP Neural Network obtin valori apropiate, in jurul valorii de 0.79, ceea ce indica faptul ca si modelele liniare sau neuronale pot functiona bine fara scalare in anumite conditii.

KNN obtine cea mai slaba performanta pe acest dataset. Acest rezultat este explicabil deoarece KNN este un algoritm bazat pe distante, iar valorile nescalate pot influenta negativ calculul vecinilor apropiati.

10.2 Compararea modelelor pe dataset-ul Normalized

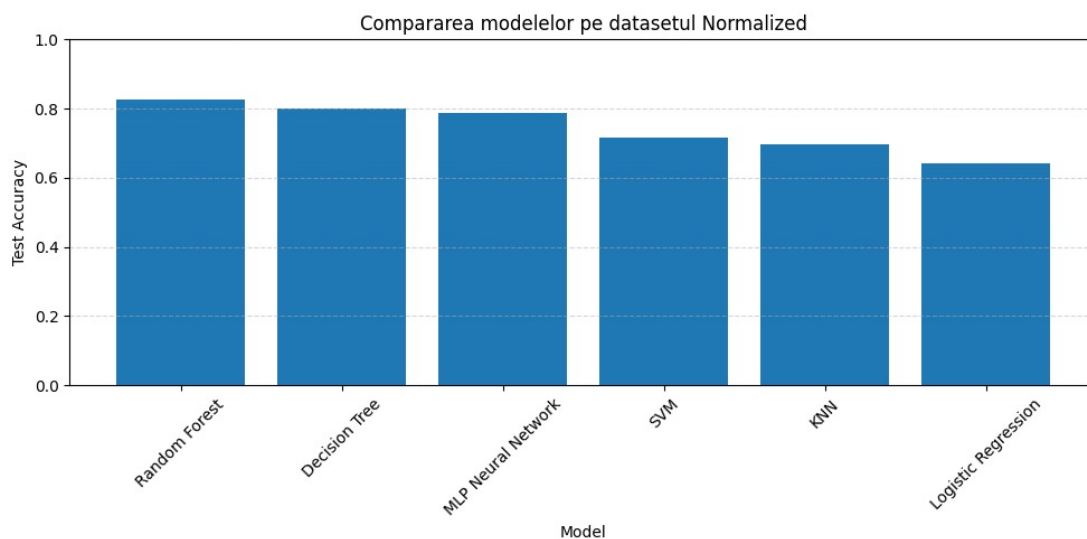


Figure 12: Compararea modelelor pe dataset-ul Normalized

Pe dataset-ul Normalized, Random Forest obtine cea mai buna performanta, depasind valoarea de 0.82. Decision Tree si MLP Neural Network obtin rezultate apropiate, in timp ce SVM si KNN au valori mai reduse.

Rezultatul confirma faptul ca Random Forest este un model robust, capabil sa obtina performante bune chiar si atunci cand datele sunt transformate prin normalizare.

10.3 Compararea modelelor pe dataset-ul MinMax Scaled

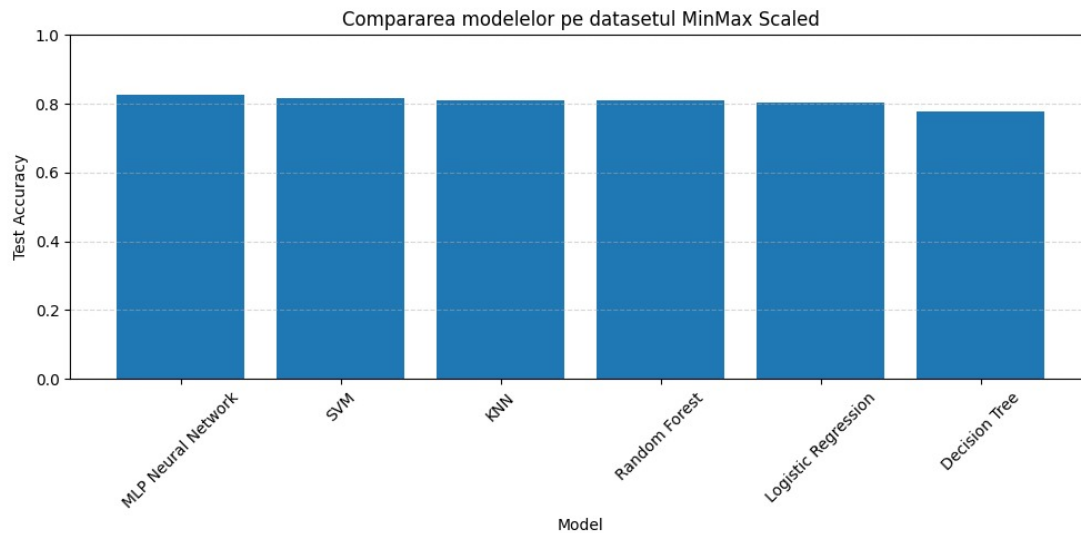


Figure 13: Compararea modelelor pe dataset-ul MinMax Scaled

Pe dataset-ul MinMax Scaled, MLP Neural Network obtine cea mai buna performanta, cu Test Accuracy de aproximativ 0.8268. Acest lucru este normal deoarece rețelele neuronale sunt sensibile la scara datelor, iar transformarea valorilor intr-un interval comun poate imbunatati procesul de antrenare.

SVM, KNN, Random Forest si Logistic Regression obtin rezultate apropiate, toate depasind valoarea de 0.80. Decision Tree are o performanta mai redusa comparativ cu celelalte modele pe acest dataset.

10.4 Compararea modelelor pe dataset-ul Standardized

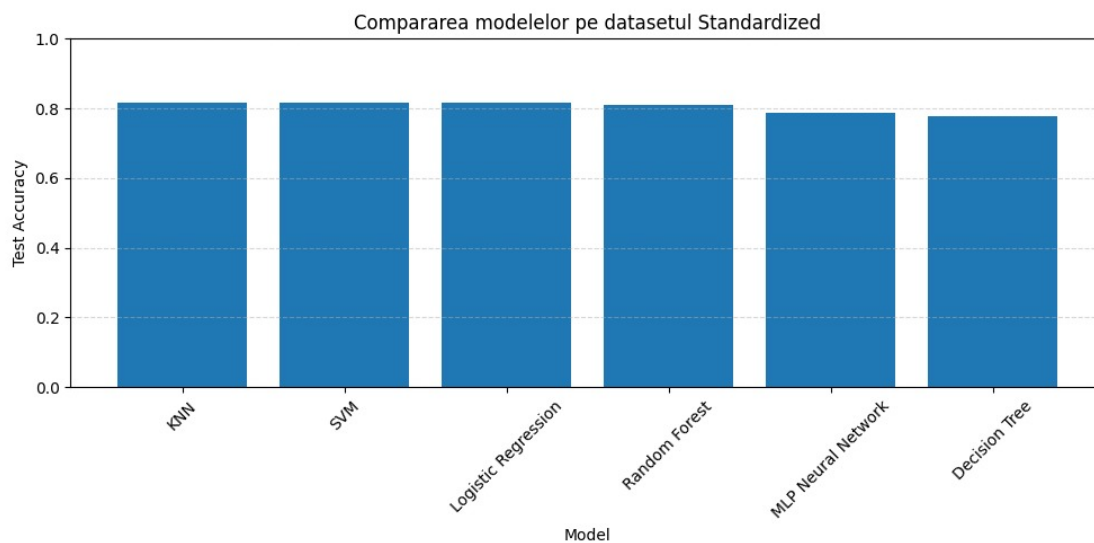


Figure 14: Compararea modelelor pe dataset-ul Standardized

Pe dataset-ul Standardized, cele mai bune rezultate sunt obtinute de KNN, SVM si Logistic Regression, toate avand valori apropiate de 0.8156. Acest rezultat arata ca standardizarea este utila pentru algoritmi sensibili la scara datelor.

Random Forest obtine o valoare apropiata de aceste modele, ceea ce confirma stabilitatea algoritmului. Decision Tree are cea mai redusa performanta in aceasta comparatie.

10.5 Compararea modelelor pe dataset-ul Robust Scaled

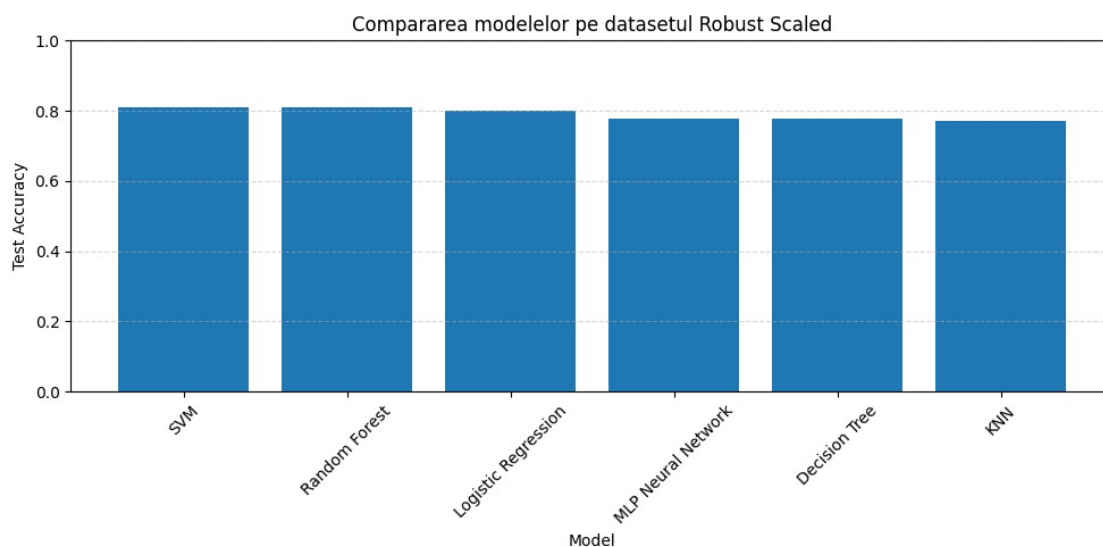


Figure 15: Compararea modelelor pe dataset-ul Robust Scaled

Pe dataset-ul Robust Scaled, SVM si Random Forest obtin cele mai bune performante, ambele in jurul valorii de 0.81. Logistic Regression are o valoare apropiata, ceea ce arata ca aceasta metoda de scalare poate fi utila pentru modele liniare.

Robust Scaling este util in special atunci cand exista valori extreme, deoarece foloseste statistici mai putin sensibile la outlieri. In acest caz, rezultatele raman apropiate intre mai multe modele, ceea ce indica o performanta stabila.

11 Analiza performantelor pentru fiecare model

In aceasta sectiune sunt analizate performantele fiecarui algoritm pe cele cinci variante ale dataset-ului.

11.1 Performanta modelului Decision Tree pe fiecare tip de dataset

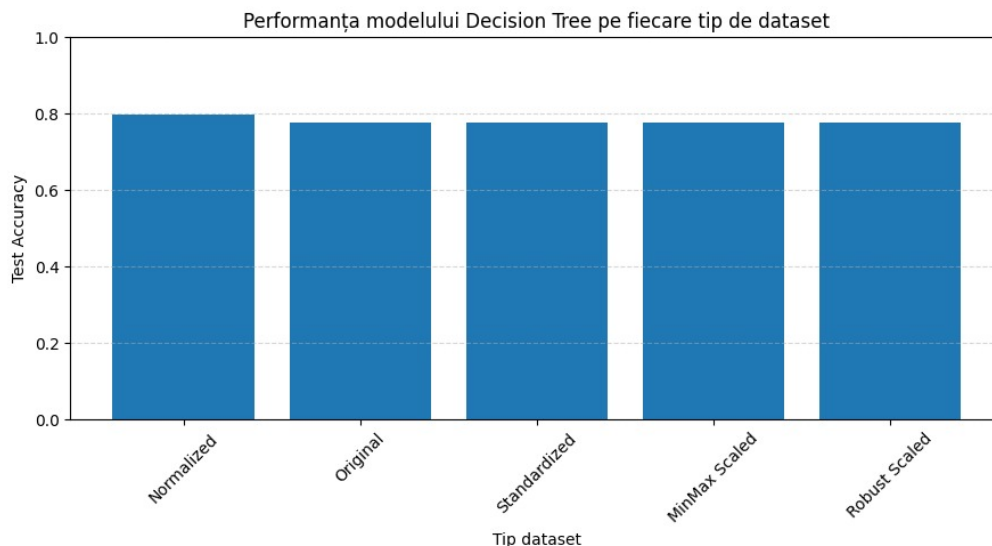


Figure 16: Performanta modelului Decision Tree pe fiecare tip de dataset

Graficul prezinta performanta modelului Decision Tree pe fiecare varianta de dataset. Se observa ca cea mai buna performanta este obtinuta pe dataset-ul Normalized, cu o valoare apropiata de 0.80.

Celelalte variante de dataset obtin valori foarte apropiate, in jurul valorii de 0.77. Acest lucru arata ca Decision Tree nu este influentat foarte puternic de scalarea datelor, deoarece arborii de decizie nu depind in mod direct de distanta dintre valori.

11.2 Performanta modelului Random Forest pe fiecare tip de dataset

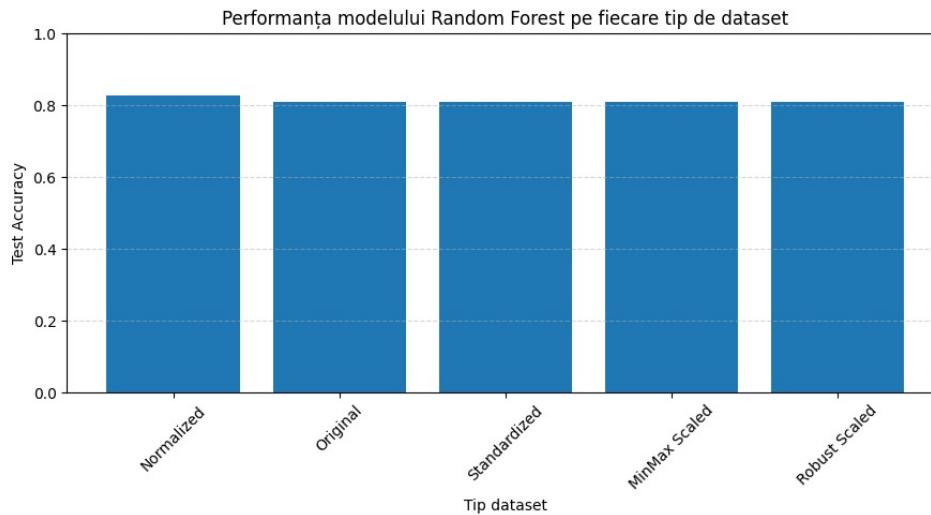


Figure 17: Performanta modelului Random Forest pe fiecare tip de dataset

Graficul arata ca Random Forest obtine cea mai buna performanta pe dataset-ul Normalized, cu o valoare Test Accuracy de aproximativ 0.8268. Pe celelalte variante ale dataset-ului, modelul obtine valori foarte apropiate, in jur de 0.81.

Acest comportament arata ca Random Forest este un model stabil si robust. Diferentele mici dintre variantele de preprocesare indica faptul ca modelul nu este foarte dependent de scalarea datelor.

11.3 Performanta modelului MLP Neural Network pe fiecare tip de dataset

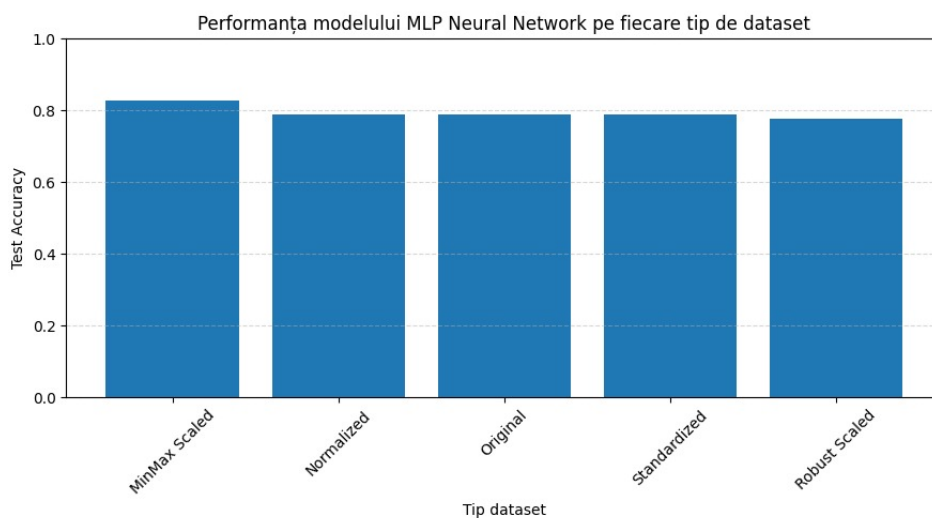


Figure 18: Performanta modelului MLP Neural Network pe fiecare tip de dataset

MLP Neural Network obtine cea mai buna performanta pe dataset-ul MinMax Scaled, cu o valoare Test Accuracy de aproximativ 0.8268. Acest rezultat este explicabil deoarece rețelele neuronale functioneaza mai bine atunci cand valorile de intrare sunt aduse la o scara comuna.

Pe celelalte variante ale dataset-ului, performanta este putin mai scazuta, dar ramane apropiata de 0.78 - 0.79. Acest lucru indica faptul ca MLP este sensibil la metoda de preprocesare.

11.4 Performanta modelului KNN pe fiecare tip de dataset

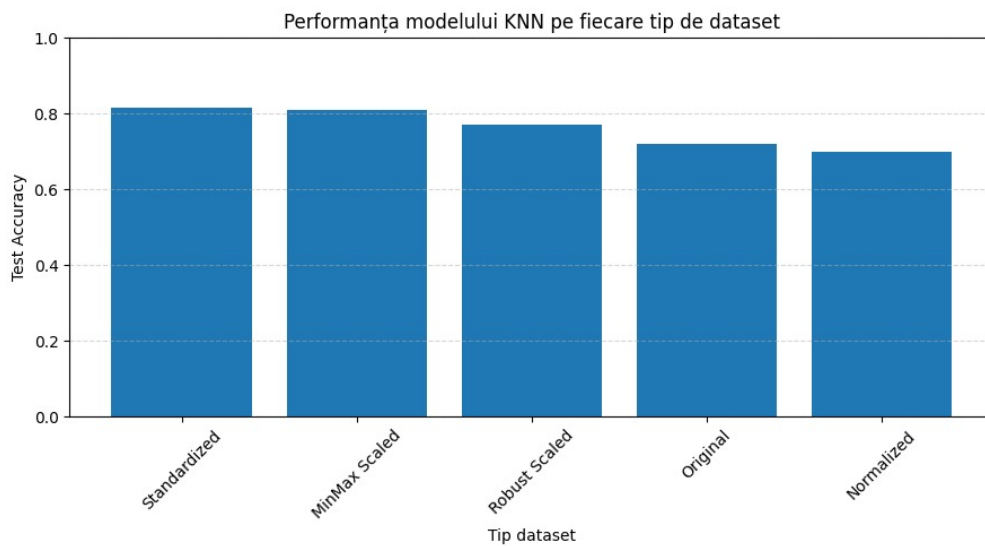


Figure 19: Performanta modelului KNN pe fiecare tip de dataset

KNN obtine cele mai bune rezultate pe dataset-urile Standardized si MinMax Scaled. Acest lucru este normal deoarece KNN se bazeaza pe distante, iar scalarea datelor influenteaza direct modul in care sunt calculati vecinii apropiati.

Performanta scade pe dataset-ul Original si pe cel Normalized, ceea ce arata ca alegerea metodei de scalare este foarte importanta pentru acest algoritim.

11.5 Performanta modelului SVM pe fiecare tip de dataset

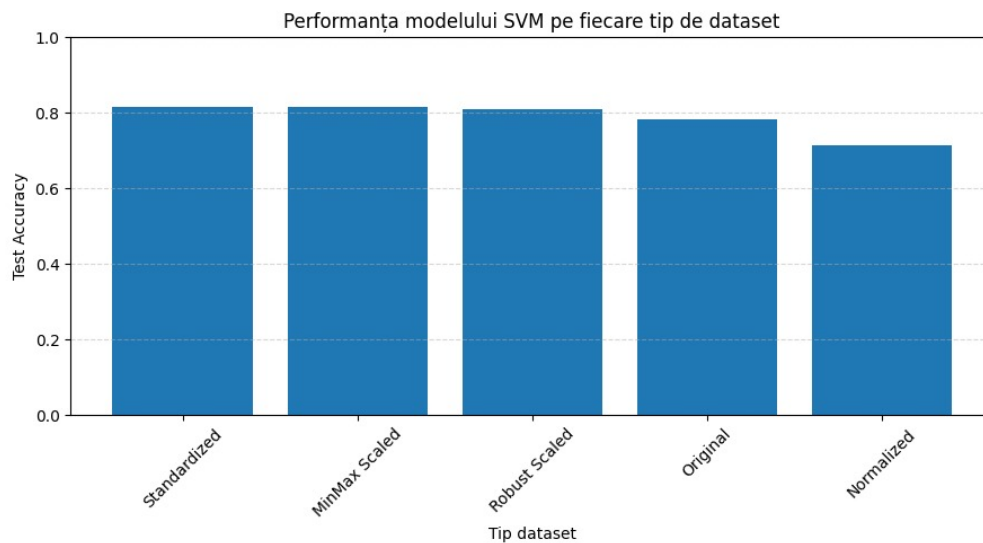


Figure 20: Performanta modelului SVM pe fiecare tip de dataset

SVM obtine cele mai bune rezultate pe dataset-urile Standardized, MinMax Scaled si Robust Scaled. Aceste valori apropiate de 0.81 indica faptul ca SVM beneficiaza de metodele de scalare.

Pe dataset-ul Normalized performanta scade, ceea ce arata ca nu orice metoda de transformare a datelor imbunatateste performanta unui model.

11.6 Performanta modelului Logistic Regression pe fiecare tip de dataset

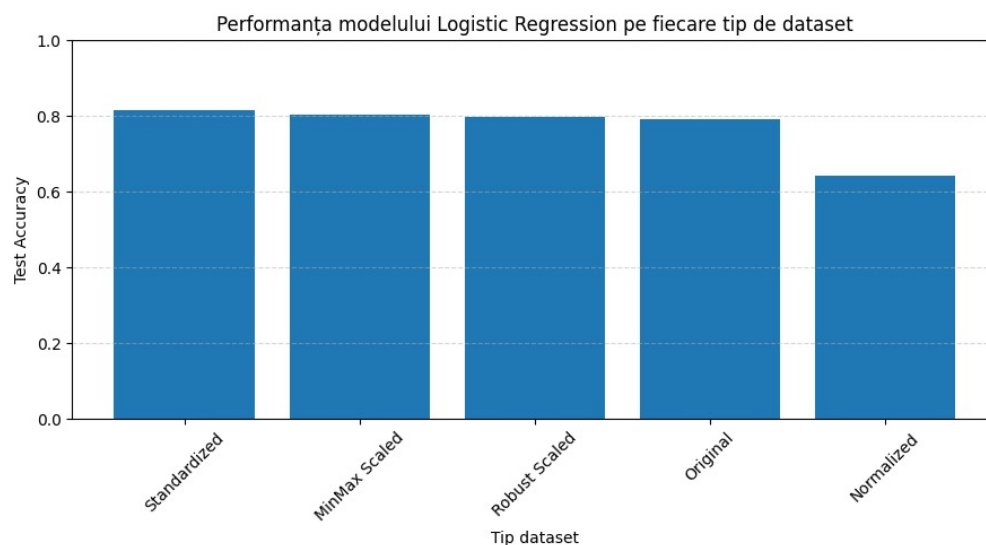


Figure 21: Performanta modelului Logistic Regression pe fiecare tip de dataset

Logistic Regression obtine cea mai buna performanta pe dataset-ul Standardized, urmat de MinMax Scaled si Robust Scaled. Acest rezultat arata ca modelul liniar beneficiaza de metodele de scalare care aduc variabilele la o distributie mai echilibrata.

Pe dataset-ul Normalized, performanta scade semnificativ, ceea ce indica faptul ca aceasta metoda de preprocesare nu este potrivita pentru Logistic Regression in acest experiment.

12 Matricea de confuzie pentru cel mai bun model

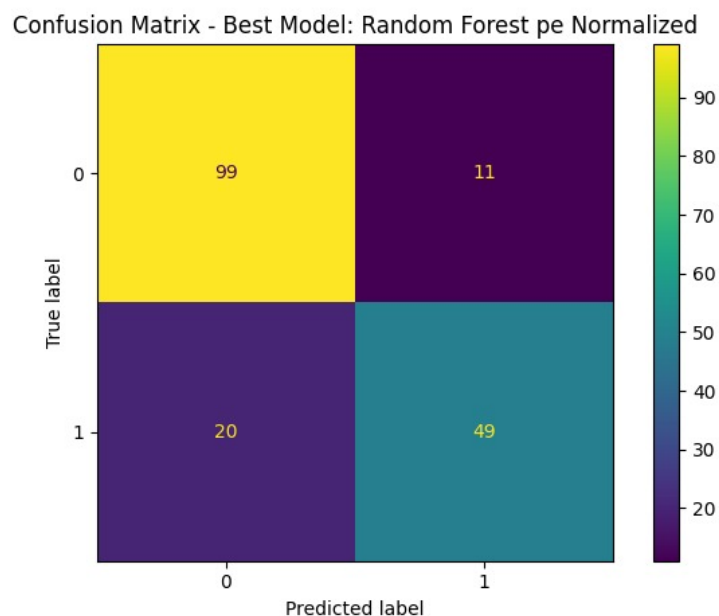


Figure 22: Matricea de confuzie pentru cel mai bun model: Random Forest pe dataset-ul Normalized

Matricea de confuzie prezinta modul in care modelul Random Forest antrenat pe dataset-ul Normalized a clasificat exemplele din setul de test. Valorile de pe diagonala principala reprezinta predictiile corecte, iar valorile din afara diagonalei reprezinta erorile de clasificare.

In acest caz, modelul a clasificat corect 99 de exemple din clasa 0 si 49 de exemple din clasa 1. De asemenea, au existat 11 exemple din clasa 0 care au fost prezise ca apartinand clasei 1 si 20 de exemple din clasa 1 care au fost prezise ca apartinand clasei 0.

Rezultatul arata ca modelul are o performanta buna in identificarea clasei 0, dar are mai multe erori in cazul clasei 1. Acest lucru explica de ce F1 Score este mai mic decat Accuracy: modelul are o acuratete generala buna, dar identificarea clasei pozitive nu este perfecta.

13 Insight-uri automate generate de sistem

```
===== INSIGHT-URI AUTOMATE =====

Cel mai bun rezultat general este obținut de modelul Random Forest pe datasetul Normalized.
Accuracy: 0.8268, F1 Score: 0.7597

Cel mai bun dataset pentru fiecare model:
- Decision Tree: cel mai bun pe Normalized cu Accuracy = 0.7989
- KNN: cel mai bun pe Standardized cu Accuracy = 0.8156
- Logistic Regression: cel mai bun pe Standardized cu Accuracy = 0.8156
- MLP Neural Network: cel mai bun pe MinMax Scaled cu Accuracy = 0.8268
- Random Forest: cel mai bun pe Normalized cu Accuracy = 0.8268
- SVM: cel mai bun pe Standardized cu Accuracy = 0.8156

Cel mai bun model pentru fiecare variantă de dataset:
- MinMax Scaled: cel mai bun model este MLP Neural Network cu Accuracy = 0.8268
- Normalized: cel mai bun model este Random Forest cu Accuracy = 0.8268
- Original: cel mai bun model este Random Forest cu Accuracy = 0.8101
- Robust Scaled: cel mai bun model este SVM cu Accuracy = 0.8101
- Standardized: cel mai bun model este KNN cu Accuracy = 0.8156

Modele cu overfitting ridicat:
- KNN pe Original are overfitting ridicat (0.2625).
- KNN pe Normalized are overfitting ridicat (0.2848).
```

Figure 23: Insight-uri automate generate pe baza rezultatelor

Pe baza rezultatelor obtinute, sistemul a generat automat o serie de observatii importante. Cel mai bun rezultat general este obtinut de modelul Random Forest pe dataset-ul Normalized, cu Accuracy de 0.8268 si F1 Score de 0.7597.

Pentru fiecare model, cele mai bune configuratii au fost:

- Decision Tree: cel mai bun pe Normalized, cu Accuracy de 0.7989;
- KNN: cel mai bun pe Standardized, cu Accuracy de 0.8156;
- Logistic Regression: cel mai bun pe Standardized, cu Accuracy de 0.8156;
- MLP Neural Network: cel mai bun pe MinMax Scaled, cu Accuracy de 0.8268;
- Random Forest: cel mai bun pe Normalized, cu Accuracy de 0.8268;
- SVM: cel mai bun pe Standardized, cu Accuracy de 0.8156.

Pentru fiecare varianta de dataset, cele mai bune modele au fost:

- MinMax Scaled: cel mai bun model este MLP Neural Network, cu Accuracy de 0.8268;
- Normalized: cel mai bun model este Random Forest, cu Accuracy de 0.8268;
- Original: cel mai bun model este Random Forest, cu Accuracy de 0.8101;

- Robust Scaled: cel mai bun model este SVM, cu Accuracy de 0.8101;
- Standardized: cel mai bun model este KNN, cu Accuracy de 0.8156.

Sistemul a identificat si doua situatii de overfitting ridicat:

- KNN pe dataset-ul Original, cu diferenta de overfitting de 0.2625;
- KNN pe dataset-ul Normalized, cu diferenta de overfitting de 0.2848.

Aceste insight-uri arata ca Random Forest este cea mai echilibrata alegere generala, in timp ce KNN poate deveni instabil atunci cand datele nu sunt scalate corespunzator sau cand normalizarea nu este potrivita pentru structura dataset-ului.

14 Cel mai bun model identificat

Pe baza rezultatelor obtinute, cel mai bun model identificat este Random Forest aplicat pe dataset-ul Normalized. Acesta obtine o valoare Test Accuracy de aproximativ 0.8268 si un F1 Score de aproximativ 0.7597.

Modelul Random Forest are si o diferenta redusa intre Train Accuracy si Test Accuracy, de aproximativ 0.0299. Aceasta diferenta indica faptul ca modelul generalizeaza bine pe datele de test si nu prezinta un risc ridicat de overfitting.

Prin comparatie, MLP Neural Network pe dataset-ul MinMax Scaled obtine aceeasi valoare pentru Test Accuracy, insa F1 Score este mai mic. Din acest motiv, Random Forest pe dataset-ul Normalized poate fi considerat cea mai echilibrata configuratie.

15 Concluzii

In cadrul proiectului a fost implementat un sistem multi-agent pentru compararea automata a algoritmilor de Machine Learning pe dataset-ul Titanic. Sistemul realizeaza intregul flux de lucru, de la incarcarea datelor pana la generarea rezultatelor si a graficelor comparative.

Utilizarea GridSearchCV a permis optimizarea hiperparametrilor pentru fiecare model, crescand sansele de obtinere a unor rezultate mai bune. De asemenea, aplicarea mai multor metode de preprocesare a permis analiza impactului scalarilor asupra performantelor.

Rezultatele au demonstrat ca performanta unui model depinde atat de algoritmul utilizat, cat si de modul in care datele sunt preprocesate. Random Forest pe dataset-ul Normalized a obtinut cea mai buna combinatie intre Test Accuracy, F1 Score si generalizare.

Analiza graficelor si a tabelor a permis identificarea celor mai bune configuratii si evaluarea riscului de overfitting. Majoritatea modelelor au generalizat bine, dar anumite

combinatii, in special unele variante ale modelului KNN, au prezentat diferente mai mari intre performanta pe train si test.

Insight-urile automate confirma faptul ca Random Forest pe dataset-ul Normalized este cea mai buna configuratie generala. In acelasi timp, analiza a evidentiat ca MLP Neural Network functioneaza foarte bine pe date scalate MinMax, iar KNN are nevoie de scalare potrivita pentru a evita overfitting-ul.

Proiectul poate fi extins prin:

- adaugarea unor noi algoritmi;
- utilizarea unor dataset-uri mai complexe;
- introducerea unor metode automate de selectie a caracteristicilor;
- dezvoltarea unor agenti suplimentari pentru interpretabilitatea modelelor.